Architectural Blueprint for "Xeno-Conquest": A Persistent Real-Time Strategy Web Application

Executive Summary

This report serves as a comprehensive technical design document, strategic roadmap, and operational manual for the development of "Xeno-Conquest," a browser-based, massive multiplayer online real-time strategy (MMORTS) game. The project vision entails a contiguous, flat 2D alien world map featuring seamless semantic zoom capabilities—ranging from planetary macro-views of territories down to tactical micro-views of individual combat units engaging in animated firefights. Unlike traditional high-frequency RTS games (e.g., *StarCraft*) or long-cycle 4X games (e.g., *Neptune's Pride*), this project targets a specific "mid-core" pacing niche: matches lasting approximately 24 to 48 hours, playable continuously across desktop and mobile devices via a unified web interface.

The analysis identifies the primary technical challenge as the dichotomy between "slow" server-side persistence and "fast" client-side visualization. The server must simulate a persistent world authoritatively over days, utilizing asynchronous job queues to manage long-duration troop movements, while the client must render engaging, high-fidelity combat animations (idle movements, shooting) when the player zooms in, despite the underlying logic operating on a slower tick rate. This requires a decoupling of simulation time from rendering time, a pattern often seen in complex simulation games but rarely applied to browser-based RTS.

Furthermore, to address the specific operational friction of AI coding assistants (such as GitHub Copilot) losing context during large-scale development, this report includes a dedicated "Context Engineering" framework. This involves strict modularization, type sharing via monorepos, and context-bounding strategies to ensure the AI remains cognizant of the broader architectural goals without hallucinating or degrading into "spaghetti code." The recommended technology stack prioritizes **React** for complex UI state management, **Pixi.js** for high-performance WebGL rendering of the map, and a **Node.js/Redis** backend using **BullMQ** for reliable, time-delayed job scheduling. This stack ensures mobile responsiveness, high frame rates for thousands of sprites, and robust state synchronization across long sessions.

---

1. Game Design and Pacing Mechanics: The "Mid-Core" Philosophy

1.1 The Pacing Gap: Defining Asynchronous Real-Time Strategy

The proposed match duration of 24 to 48 hours places "Xeno-Conquest" in a unique and under-served pacing category. The strategy game market is currently bifurcated. On one end, session-based RTS games require intense, unbroken focus for 15 to 45 minutes, demanding high Actions Per Minute (APM). This alienates mobile users who play in short bursts and cannot commit to uninterrupted sessions. On the other end, "Grand Strategy" or persistent browser games often span weeks or months. While these accommodate casual play, they suffer from high player attrition due to the sheer length of commitment required to see a victory condition. Research indicates a significant gap for "Asynchronous Real-Time" gameplay that fits between these extremes.

In this model, the game progresses continuously in real-time, but actions such as movement and construction take significant durations—ranging from 30 minutes to several hours. This design allows players to queue orders during short sessions (5–10 minutes) on their mobile devices during a commute or break, and return later to see the results. It caters to the "busy parent" or "office worker" demographic who desire the depth of strategy without the twitch reflexes or the months-long grind. However, this pacing presents a design risk: if the game is too slow, it becomes boring; if it is too fast, players feel anxious about sleeping.

Tick-Based vs. Interpolated Time Systems

To achieve this specific pacing, the game cannot run on a standard 60 FPS physics loop server-side, as maintaining that frequency for a 48-hour match is computationally wasteful and unnecessary. Instead, the architecture must utilize a discrete Tick System combined with Client-Side Interpolation.

- **Server Tick:** The server processes logic at a slow, deliberate rate (e.g., every 10 seconds or essentially "on event"). Game state changes are transactional: a unit arriving at a destination is an event, not a continuous physics calculation.
- **Client Interpolation:** The client receives the state intent ("Unit A moves from Territory X to Y, departing at 12:00, arriving at 14:00"). The client then *interpolates* this movement smoothly on the screen. If the player zooms in, they see the unit walking across the terrain. If they close the browser and check back an hour later, the unit has visually progressed halfway across the map. This creates the illusion of a living, breathing world without requiring the server to update positions millisecond-by-millisecond.1

1.2 Territory and Map Dynamics: The Alien Landscape

The map is described as a flat 2D alien world divided into territories, distinct from Earth. This implies a need for procedural generation that creates organic, non-uniform shapes. The standard square grid (tilemap) is often insufficient for conveying organic "territory." Instead, a **Voronoi Diagram** or **Mesh-based** territory generation system is recommended. This divides

the map into polygonal regions based on "seed" points (stars, cities, or resource nodes), creating natural-looking borders that resemble political maps.

- **Territory Control & Resource Generation:** Owning territory is the primary economic driver. Rather than harvesting individual resource patches (like *StarCraft*), the territory itself generates resources (troops, currency, energy) over time. This shifts the player's focus from micro-managing workers to macro-managing frontlines.
- **Topography as Strategy:** The "Alien" nature of the map allows for unique terrain features. "Water" acts as a hard barrier or requires specific tech (transports) to cross. "Mountains" or "Craters" could create choke points. The map generation algorithm must ensure these features interact logically—water bodies should aggregate in low "elevation" areas, even on a flat 2D map, to create strategic peninsulas and islands.
- **The "Frontline" Dynamic:** In a 2-day match, the frontline should not shift instantly. Taking a territory might require a "Siege Phase" or "Occupation Timer" of 1–2 hours. This is critical for the "Real World Time" aspect. If a territory flips instantly, a player sleeping for 8 hours could lose their entire empire. A Siege mechanic gives the defender time to receive a push notification ("Your territory is under siege!") and react, reinforcing the asynchronous nature of the multiplayer interaction.3

1.3 Combat Visualization: The "Zoom" Requirement

The user has a specific requirement for high-fidelity visuals: zooming in to the closest level reveals "actual troops" with idle movement, walking animations, and shooting. This creates a significant technical decoupling challenge. The server simulation is abstract (numbers vs. numbers), but the client rendering must be visceral (sprites vs. sprites).

Semantic Zoom Levels

The solution lies in "Semantic Zoom," where the data representation changes based on the camera's distance from the ground.5

1. **Macro View (Zoom Level 0–5):** The player sees the strategic situation. Territories are colored polygons. Troops are aggregated into single icons or badges (e.g., a shield icon with "50" inside). Movement is depicted as abstract arrows or lines.
2. **Region View (Zoom Level 5–12):** As the player zooms in, textures fade in. The alien terrain becomes detailed. The "50" badge might split into five "squad" icons, giving a better sense of distribution.
3. **Micro/Tactical View (Zoom Level 13+):** This is where the renderer swaps the "Count Icon" for individual "Sprite Instances." If there are 50 troops, 50 individual sprites appear. They play idle animations (checking weapons, looking around).
    - **Deterministic Combat Playback:** When combat occurs, the server calculates the result mathematically (e.g., 100 troops attack 50 defenders = Attacker wins with 60

remaining). The server sends this *result* along with a random *seed* to the client. The client uses this seed to play out a deterministic "battle movie." Unit A fires at Unit B. Unit B explodes. The visual outcome matches the server's math perfectly, but the player gets the satisfaction of watching the firefight. This "movie" approach decouples the heavy rendering from the game logic, ensuring the browser doesn't crash trying to simulate physics for thousands of bullets.6

1.4 Bot Ecology: Filling the World

To make the map feel alive, especially in the early stages of a match or in lower-population games, "Bots" are required. These cannot be simple scripts; they must play by the same rules as humans to be fair.

- **Utility AI:** Rather than complex neural networks, which are computationally expensive, bots should use Utility AI. The bot evaluates a "score" for every potential action.
  - *Score = (Value of Target Territory) / (Distance + Enemy Strength).*
  - The bot periodically checks this score for its border units and issues move commands if the score exceeds a threshold. This logic runs on the server, ideally in a separate "Bot Worker" thread to avoid lagging the main game loop.
  - **Personality Profiles:** Different bots can have different utility curves. An "Aggressive" bot weights *Enemy Strength* lower (ignoring risk). A "Defensive" bot weights *Own Territory Safety* higher. This creates variety in the opponents.8

---

2. Technical Architecture and Tech Stack

To prevent the "spaghetti code" that confuses AI assistants and leads to project failure, the architecture must be strictly layered. The separation of concerns is not just a coding best practice; it is a survival mechanism for complex browser games.

2.1 Recommended Technology Stack

Component	Technology	Rationale & Integration Strategy
Frontend Framework	**React**	Strategy games are UI-heavy. React excels at managing complex state for build queues, tech trees, chat windows, and diplomatic menus. Its component-based architecture allows for rapid iteration of the "HUD" (Heads-Up Display) that sits *over* the game map.10
Game Engine (2D)	**Pixi.js** (+ `@pixi/react`)	Pixi.js is the industry standard for high-performance 2D WebGL rendering. It is capable of rendering

Component	Technology	Rationale & Integration Strategy
		10,000+ sprites (troops) via batching, which is essential for the "Zoom in" requirement. Unlike heavier engines like Godot or Unity, Pixi.js has a tiny footprint, ensuring fast load times on mobile networks. The `@pixi/react` library allows declaring Pixi objects as JSX components, bridging the gap between the UI and the Canvas.10
Build Tool	**Vite**	Vite provides extremely fast HMR (Hot Module Replacement). When tweaking the "zoom speed" or "sprite color," Vite updates the browser almost instantly, which is crucial for fine-tuning the game feel.14
Language	**TypeScript**	**Mandatory.** Shared types (interfaces) between Backend and Frontend allow the AI (Copilot) to understand data structures across the entire stack. Without TypeScript, the AI loses context of what a "Unit" object looks like when moving between server and client code.15
Backend Runtime	**Node.js**	Its event-driven, non-blocking I/O architecture is perfectly suited for the WebSocket nature of real-time games. It shares the JavaScript/TypeScript language with the frontend, simplifying code sharing.
Communication	**Socket.IO**	Provides robust, real-time bidirectional communication. It handles "Room" logic (e.g., chat rooms, match lobbies) and automatic reconnection strategies out of the box, which is vital for mobile devices that frequently switch between Wi-Fi and 4G.17
Job Queue	**BullMQ** (Redis)	Critical for "slow" mechanics. Handling "Troops arrive in 45 minutes" is best done via persistent job queues. Using in-memory `setTimeout` is fragile; if the server restarts, the timer is lost. BullMQ persists these jobs in Redis, ensuring gameplay reliability.19
Database	**PostgreSQL**	Relational data integrity for user accounts, match history, and the cold storage of game state.
Cache/State	**Redis**	High-speed in-memory storage for the "Hot" game state (current unit positions, territory ownership).

2.2 Monorepo Structure: The Context Container

Using a monorepo allows the sharing of TypeScript interfaces ("DTOs" - Data Transfer Objects). If you define `interface Troop { health: number; x: number; y: number; }` in a shared folder, both the server logic and the Pixi renderer use it. This drastically reduces AI hallucination because the data shape is constant across the codebase.

Proposed Directory Structure:

/xeno-conquest-monorepo

├── package.json

├── /packages

│ └── /shared # THE TRUTH. Contains all TypeScript Interfaces, Constants, and Game Configs.

│ ├── types.ts # interface Unit, interface Territory

│ ├── constants.ts # GAME_SPEED, TILE_SIZE, ZOOM_LEVELS

│ └── maps.ts # Algorithms for procedural generation shared by client/server

├── /apps

│ ├── /client # React + Pixi.js (Vite). The visual layer.

│ ├── /server # Node.js + Socket.IO API. The traffic cop.

│ └── /worker # BullMQ Job Processor. The game logic engine.

This structure ensures that when the "Worker" calculates a battle result, it uses the exact same CombatResult interface that the "Client" uses to render the explosion.

---

3. Client-Side Engineering: The Infinite Zoom Map

The visual core of "Xeno-Conquest" is the map. Implementing a "Google Maps" style zoom that transitions seamlessly from abstract territories to animated sprites requires specific rendering techniques and optimizations to run smoothly on mobile devices.

3.1 The Infinite Canvas Pattern

The map must function as an "Infinite Canvas." This involves translating screen coordinates (pixels on the mobile screen) to world coordinates (the game space).

- **The Viewport Library:** We will utilize `pixi-viewport`. This library handles the complex math required for:
  - **Panning:** Dragging the map with a finger or mouse.
  - **Pinching:** Zooming in and out on touch devices.
  - **Deceleration:** Adding "weight" to the map so it glides to a stop after a flick, giving it a premium feel.
  - **Clamping:** Preventing the player from panning into the void off the edge of the map.

3.2 Spatial Partitioning with Quadtrees

The user mentions "Take over the map" with potentially thousands of units. If there are 5,000 units on the map, iterating through them every single frame to check `if (unit.visible)` is computationally expensive and will drain mobile battery life. To solve this, we implement a **Quadtree**.

- **The Algorithm:** A Quadtree recursively divides the 2D world into four quadrants.
- **Insertion:** All static objects (Territories, Resource Nodes) and dynamic objects (Units) are inserted into the Quadtree based on their X/Y coordinates.
- **Querying:** On every frame update (the render loop), the engine defines a "Bounding Box" that matches the current camera view (plus a small buffer margin). It queries the Quadtree: "Give me all objects inside this box."
- **Culling:** The renderer *only* processes and draws the objects returned by this query. A unit on the other side of the planet is ignored entirely by the GPU. This technique ensures that performance depends on *how many units are on screen*, not how many units exist in the total world.21

3.3 Implementing "Semantic Zoom" (LOD)

This feature creates the "cool factor" the user desires. It requires a system that monitors the `viewport.scale` and swaps assets accordingly.

Technical Implementation Strategy:

We define discrete LOD (Level of Detail) tiers.

- **LOD 0 (Scale < 0.2):** The "Strategic View."
  - *Rendering:* Simplified geometry. Territories are drawn as solid colors using `PIXI.Graphics`.
  - *Units:* No sprites are rendered. Instead, a "Cluster Icon" (e.g., a circle with a number) is drawn at the center of the territory.
- **LOD 1 (Scale 0.2 - 0.8):** The "Operational View."

    - *Rendering:* High-resolution terrain textures fade in (using shaders or alpha blending). Borders become distinct.
    - *Units:* The Cluster Icons fade out. Individual "Dot" sprites appear, representing squads.
- **LOD 2 (Scale > 0.8):** The "Tactical View."
    - *Rendering:* Full detail.
    - *Units:* The "Dots" are replaced by `PIXI.AnimatedSprite` instances.
    - *Optimization:* We enable "Instanced Rendering" (using Pixi's `ParticleContainer` or v8 `RenderGroup`). This allows the GPU to draw 1,000 soldiers in a single draw call, provided they share the same base texture (spritesheet). This is crucial for maintaining 60 FPS on mobile devices during large battles.24

3.4 The React-Pixi Bridge: Managing Complexity

A common mistake is attempting to make every single map sprite a React component. React's "Reconciliation" process (checking what changed in the DOM) is too slow for a game loop running at 60 FPS with thousands of entities.

- **The Hybrid Approach:** We use React for the `<Stage>` (the canvas container) and the UI overlays (HTML menus). However, the internal contents of the map—the thousands of units—are managed by a custom TypeScript class, let's call it `MapManager`.
- **Communication Flow:**
    - React receives a WebSocket update: "Unit moved."
    - React does *not* re-render the map component.
    - React calls `MapManager.updateUnit(id, newPosition)`.
    - `MapManager` directly manipulates the Pixi sprite properties.
    - This bypasses React's overhead for high-frequency updates, keeping the game buttery smooth.14

---

4. Backend Architecture: The Time Engine

The backend defines the "Real World Time" aspect. It must be authoritative—players send *intent*, and the server validates and executes it over time.

4.1 The Job Queue Pattern (BullMQ)

For a game taking 1-2 days, creating a `while(true)` loop on the server to update every unit is inefficient and prone to lag. Instead, we use an **Event-Driven Job Queue**.

Workflow Example: Moving Troops

1. **Client Intent:** The player selects 50 units and clicks "Move to Territory B". The client emits `CMD_MOVE_UNIT` { unitId, targetX, targetY }.
2. **Server Validation:** The API checks: Does the player own these units? Are they alive? Is the destination reachable?
3. **Calculation:** The server calculates travel time. Distance = 100km, Speed = 10km/h. Duration = 10 hours. Arrival Timestamp = `Now + 10 hours`.
4. **Queueing:** The server adds a job to **BullMQ**: `queue.add('processArrival', { unitId, targetId }, { delay: 36000000 })` (10 hours in ms).
5. **Persistence:** The server updates the unit's state in Redis to `status: "MOVING"`, `arrivalTime: <timestamp>`, `startPosition: <coord>`, `targetPosition: <coord>`.
6. **Broadcast:** The server emits `UNIT_MOVING` to all connected clients.
7. **Client Visualization:** The client receives the event. It doesn't need to ping the server. It knows the start time, end time, and start/end positions. It uses linear interpolation (Lerp) to draw the unit at the correct spot on the line, regardless of when the player looks at it.
8. **Execution:** 10 Hours later, BullMQ fires the job. The "Worker" server wakes up, processes the arrival (e.g., starts combat or captures territory), updates the database, and sends a new `UNIT_ARRIVED` event.20

4.2 Handling "Real-Time" Combat

When units *do* clash, the user wants to see shooting.

- **Server Side:** Combat is calculated in "rounds" (e.g., every 10 seconds of the fight). The server calculates damage and updates HP in Redis.
- **Client Side:** The client subscribes to the `combat_update` socket channel. It receives: `{"attacker_id": 1, "target_id": 2, "damage": 5}`. The client then triggers the `shoot` animation on Unit A and the `damage` blink on Unit B. This keeps the visual synced with the "slow" server logic.2

4.3 Persistence and Crash Recovery

Since matches last days, the server *will* restart or crash at some point. The architecture must be resilient.

- **Redis (Hot State):** Stores active game state (unit locations, current health) for fast read/write access.
- **PostgreSQL (Cold State):** Every X minutes (or on major events like player disconnects), the Redis state is flushed to Postgres. This provides a hard backup.
- **Rehydration:** On server restart, the system reads from Postgres, repopulates Redis, and re-initializes the BullMQ jobs based on stored timestamps. The game picks up exactly where it left off.29

5. Game Design Mechanics

5.1 Pacing and Sleep Mechanics

The user noted a gap for "quicker paced" versions of long strategy games. However, a 48-hour continuous real-time game creates a physiological problem: sleep. If a player can be destroyed while they sleep, they will quit.

- **The "Night Truce" or "Shield" Mechanic:** Players can activate a "Shield" for 8 hours (once per match) to sleep. Alternatively, movement speed or combat effectiveness can be drastically reduced during a global "Night" cycle, making attacks harder to execute while users are offline. This respects the user's life outside the game.3
- **Push Notifications:** Essential for this pacing. "Your territory is under attack!" allows players to disengage and only return when critical events happen.

5.2 Fog of War and Scouting

To encourage the "Zoom" mechanic, information must be hidden. Players should only see enemy unit counts in adjacent territories. To see deeper into enemy land, they must send scouts. This forces players to physically pan and zoom around the map to gather intelligence, making the "Map" a gameplay tool, not just a background.30

5.3 Bot AI Architecture

Bots are required to fill the map.

- **Avoid Complex Neural Networks:** They are too expensive for thousands of units.
- **Use Utility AI (GOAP):** The bot evaluates a score for actions.
  - *Score = (Value of Territory) / (Distance + Enemy Strength).*
  - The bot periodically checks this score for its units and issues move commands.
  - This logic runs on the server, perhaps in a separate "Bot Worker" thread to avoid lagging the main game loop.8

---

6. Copilot Context Strategy: Preventing "Lost in the Sauce"

The user specifically noted that Copilot fails due to scope confusion. AI Coding Assistants work on a "Context Window." If the project is too big, they "forget" the rules. To fix this, you must explicitly manage the context using a dedicated file.

6.1 The `copilot-context.md` File

Create a file at `.github/copilot-instructions.md` (or `copilot-context.md` in the root). Copilot often reads this file to ground its responses.

Content of the file:

Xeno-Conquest Project Context

Core Identity

- **Genre:** Persistent Browser MMORTS (24-48h matches).
- **Visual Style:** Top-down 2D, Sci-fi Alien World.
- **Key Mechanic:** Seamless semantic zoom (Map -> Icons -> Animated Sprites).

Tech Stack Rules

- **Frontend:** React (UI Overlay) + Pixi.js (Game View) + Vite.
- **Backend:** Node.js (Express) + Socket.IO + BullMQ (Redis).
- **Language:** Strict TypeScript.
- **State:** Zustand (Client), Redis (Server).

Architecture Patterns

1. **The "Manager" Pattern:** React components do NOT control Pixi Sprites directly. React calls methods on a `GameMapManager` class, which manipulates the Pixi Stage.
2. **Shared Types:** Always define data structures (Unit, Territory) in `packages/shared` before implementing logic.
3. **Job Queue:** Any logic taking > 5 seconds (movement, building) MUST use the BullMQ job queue, not `setTimeout`.
4. **Optimistic UI:** Clients predict successful moves immediately; Server validates asynchronously.

Coding Style

- Use Functional Components for React.
- Use `async/await` for asynchronous operations.
- Prefer `interface` over `type` for object definitions.

6.2 Context Bounding (The "One File" Rule)

When asking Copilot for help, do not ask "How do I build the map?" (Too broad).

- **Bad Prompt:** "Make the units fight."

- **Good Prompt:** "I am working on `UnitRenderer.ts`. Using the `Unit` interface from `@shared/types`, write a function that takes a `combatState` object and plays the `shoot` animation on the `PIXI.AnimatedSprite`."

6.3 The "Summary" for Copilot

The user requested a short summary to feed Copilot. Here is the optimized text block to paste at the start of new AI sessions:

> Project Context: "Xeno-Conquest" is a persistent browser RTS (24-48h matches).
>
> Stack: React (UI), Pixi.js (Map Renderer), Node.js/Socket.IO (Server), BullMQ (Async Game Loop).
>
> Core Mechanic: Infinite map with Quadtree culling. Semantic zoom: Icons at low zoom, Animated Sprites at high zoom.
>
> Architecture:
>
> 1. **Shared:** Types defined in `/packages/shared`.
> 2. **Client:** `GameMap.tsx` initializes `MapEngine.ts` (Pixi). React overlays HTML menus.
> 3. Server: Authoritative state. Movement uses delayed Job Queues. Combat is deterministic.
>
> Current Task:

---

7. Implementation Roadmap and Focus File

To keep the project on track, maintain a `TODO.md` file. This prevents "feature creep."

Phase 1: The Skeleton (Week 1-2)

- [ ] Set up Monorepo (Turborepo or NPM Workspaces).
- [ ] Initialize React + Vite + Pixi.js boilerplate.
- [ ] Create a simple "Green Square" map in Pixi.js.
- [ ] Implement Pan and Zoom (using `pixi-viewport`).
- [ ] **Milestone:** You can pan/zoom an empty map on your phone.

Phase 2: The Data Layer (Week 3-4)

- [ ] Define `interface Territory` and `interface Unit` in Shared package.
- [ ] Set up Node.js + Socket.IO server.

- [ ] Implement "Handshake": Client connects, Server sends Map Data (JSON), Client renders Territories.
- [ ] **Milestone:** You see the generated map from the server on the client.

Phase 3: The "Slow" Movement (Week 5-6)

- [ ] Implement BullMQ on server.
- [ ] Create "Move" command: Click Unit -> Click Target -> Emit Socket Event.
- [ ] Server adds Job (e.g., 10 second delay for testing).
- [ ] Client receives "Moving" state and draws a line.
- [ ] **Milestone:** Units move and arrive at destinations persistently.

Phase 4: The Visual Polish (Week 7-8)

- [ ] Implement LOD System: Check `viewport.scale` on every frame.
- [ ] If `scale > X`, hide Icons, show Sprites.
- [ ] Add "Idle" and "Walk" animations to Sprites.
- [ ] **Milestone:** Smooth transition from dots to soldiers.

Phase 5: Combat & Gameplay (Week 9+)

- [ ] Implement Combat Logic on Server (HP deduction).
- [ ] Send Combat Events to Client.
- [ ] Trigger "Shoot" animation when event received.
- [ ] Win Condition (Take over map).

Section 1: Introduction to Asynchronous Real-Time Strategy

1.1 Defining the "Slow RTS" Genre

The user’s concept occupies a sophisticated middle ground in the strategy genre spectrum. To understand the architectural requirements, we must first classify the gameplay loop strictly.

- **Traditional RTS (StarCraft, Age of Empires):** High-frequency, sub-second decision loops (APM - Actions Per Minute). Networking relies on "Lockstep" deterministic simulation. Match duration: 15–45 minutes. *Incompatible with the user's "1-2 day" goal.*
- **Grand Strategy / 4X (Neptune’s Pride, Travian):** Low-frequency, hours/days decision loops. Gameplay is often "Spreadsheet-like" with abstract visuals. Networking is

transactional (REST/HTTP). Match duration: Weeks to Months. *Too slow for the user's "quicker paced" desire.*

- **The Target: "Mid-Core Persistent RTS":**
  - **Decision Loop:** Minutes to Hours.
  - **Simulation:** Continuous (Real-time), not Turn-Based.
  - **Visuals:** High-fidelity (animations, zoom) usually reserved for Fast RTS.
  - **Networking:** Hybrid (Socket.IO for presence/combat, Job Queues for long-duration movement).

Strategic Implication: The core design friction is "The Wait." In a 10-minute game, watching a unit walk is fun. In a 2-day game, watching a unit walk for 4 hours is boring.

Design Solution: The "Zoom" mechanic is the bridge. The "Global View" acts as the strategic dashboard (waiting, planning), while the "Zoomed View" acts as the tactical gratification (watching the specific battles or idle animations to confirm the "reality" of the world).

1.2 The "Zoom" Mechanics: Semantic Level of Detail

The user wants to zoom from a "whole map" view down to "seeing actual fight with guns." This is technically known as **Semantic Zoom**. It is not merely scaling an image (Optical Zoom); it is changing the *data representation* based on the camera distance.

Zoom Level	Visual Representation	Data Requirement	Interaction
Galaxy (0-20%)	Colored Polygons (Territories). Icons for Army Strength (e.g., "50k").	GeoJSON / Vector Data. Aggregated stats.	Strategic Move Orders (Army to Territory).
Region (20-60%)	Terrain textures (craters, rivers). Unit Clusters (Icons representing squads).	Texture Atlases. Localized unit counts.	Tactical Positioning.
Tactical (60-100%)	Individual Sprites (Soldiers, Tanks). Idle/Attack Animations. Projectiles.	Sprite Sheets. Entity-Component System (ECS).	Micro-observation (Watching the fight).

Technical Ripple Effect: This requirement dictates the rendering engine. The DOM (HTML `<div>` elements) cannot handle the transition to Level 3 efficiently on mobile. **WebGL** is mandatory.

1.3 The "Lost in the Sauce" Problem

The user explicitly mentions previous failures with AI (Copilot) getting confused. This is a symptom of **Context Overload**.

- **Cause:** RTS games involve tightly coupled systems (Networking UI Game Logic Rendering). When a user asks an AI to "add shooting," the AI doesn't know if it should modify the Server Combat Loop, the Client Network Listener, or the Sprite Renderer.
- **Solution:** The "Comprehensive Plan" provided in this report will act as a **Context Anchor**. By breaking the system into distinct "Black Boxes" (e.g., The Map Renderer is a black box that just takes `Unit` data), the user can prompt the AI to work on *one box at a time*, preventing scope creep.

Section 2: Technical Architecture & Tech Stack

2.1 Technology Selection Rationale

The Frontend: React + Pixi.js

This combination is the industry "Gold Standard" for complex 2D browser games that require both high-performance graphics and complex UIs.10

- **React:** Strategy games are 50% menus (Build queues, Tech trees, Chat, Diplomatic messages). Building these in a game engine (like Unity or Godot) is painful and often non-responsive on mobile. React excels here.
- **Pixi.js:** A fast, lightweight 2D WebGL renderer.
  - *Why not Godot?* Godot's HTML5 export is heavy (WASM blob is large), leading to slow load times on mobile data. It also struggles with the "DOM overlay" integration compared to a native JS library.12
  - *Why not Phaser?* Phaser is excellent, but Pixi.js is often preferred when integrating deeply with React, as it is strictly a rendering engine, allowing React to handle the game loop and state more flexibly.
- `@pixi/react`: A library that lets you write Pixi code using JSX syntax (e.g., `<Container><Sprite /></Container>`), bridging the two technologies.33

The Backend: Node.js + Socket.IO + Redis

For a persistent game involving timers (e.g., "Build complete in 2 hours"), the backend must be robust.

- **Node.js (TypeScript):** Allows sharing code (interfaces/types) with the frontend.

- **Socket.IO:** Handles the "Real-Time" aspect. When a fight starts, the server pushes packets to clients to trigger animations.
- **Redis + BullMQ:** This is the *critical* component for a "Slow RTS."
  - *Role:* It manages the time. When a player moves a unit, the server doesn't "sleep." It creates a **Job**.
  - `queue.add('arrival', { unitId: 123 }, { delay: 7200000 })` // Delay 2 hours.
  - If the server restarts 1 hour later, Redis persists the job. The game state is safe.19

2.2 Project Structure (Monorepo)

To keep the AI assistant focused, the project should be structured as a **Monorepo**. This strictly separates concerns.

```
/xeno-conquest
├── package.json
├── /packages
│ ├── /shared # Shared Types & Constants
│ │ ├── types.ts # export interface Unit { id: string; x: number;... }
│ │ ├── constants.ts # export const UNIT_SPEED = 10;
│ │ └── maps.ts # Procedural generation configs
├── /apps
│ ├── /client # React + Pixi.js (Vite)
│ │ ├── /src
│ │ │ ├── /components/ui # HTML Overlays (Menus)
│ │ │ ├── /components/game # Pixi Components (Map, Units)
│ │ │ └── /stores # State (Zustand/Redux)
│ ├── /server # Node.js API & Socket Server
│ │ ├── /src
│ │ │ ├── /handlers # Socket Event Handlers
│ │ │ ├── /state # Redis State Managers
```

│ │ │ └── /engine # Combat Logic

│ └── /worker # BullMQ Job Processor (The "Timekeeper")

│ ├── /src

│ │ └── /jobs # Arrival logic, Production logic

Why this helps Copilot: When you are working in `/client`, you can tell Copilot: "Import the `Unit` type from `@xeno/shared`." Copilot immediately understands the data structure without you needing to paste it, reducing errors.15

Section 3: Engineering the Map & Zoom

3.1 The Infinite Canvas Pattern

The map is an "Infinite Canvas." Implementing this requires translating screen coordinates (pixels) to world coordinates (game space).34

The Viewport Library:

Use pixi-viewport. It handles the math for:

- Dragging (Panning)
- Pinching (Zooming on mobile)
- Decelerating (Inertia after a flick)
- Clamping (Preventing panning off the map)

3.2 Quadtrees for Performance

The user mentions "Take over the map" with potentially many units. If there are 5,000 units on the map, iterating through them every frame to check `if (unit.visible)` is too slow for mobile.

Algorithm:

1. **Insert** all static objects (Territories) and dynamic objects (Units) into a Quadtree structure.
2. **On Frame Update:**
   - Get the current Viewport bounding box (e.g., `x: 0, y: 0, width: 800, height: 600`).
   - Query the Quadtree: `tree.query(viewportRect)`.
   - The result is a list of *only* the objects currently visible.

- Render only those objects. This technique (Spatial Partitioning) ensures that even if the world is massive, the phone only processes what is on screen.21

3.3 Implementing the "Deep Zoom" (LOD)

This is the user's specific "cool factor" request.

Technical Implementation:

TypeScript

```
// Pseudo-code concept for AI context
function onZoomChange(newScale: number) {
  if (newScale < 0.5) {
    // Zoomed OUT (Macro)
    unitIconContainer.visible = true;
    unitSpriteContainer.visible = false;
    territoryLayer.texture = lowResTexture;
  } else {
    // Zoomed IN (Micro)
    unitIconContainer.visible = false;
    unitSpriteContainer.visible = true;
    territoryLayer.texture = highResTexture;

    // Only start animations when zoomed in to save battery
    startIdleAnimations(visibleUnits);
  }
}
```

Optimizing the "Fight":

When the user zooms into a "Fight," we cannot simulate bullets using physics (Box2D) because it's too expensive for thousands of units and hard to sync.

Visual Trickery:

- Units don't actually aim. They play a `Shoot_Animation` directionally towards the enemy.
- Muzzle flash sprites appear.
- "Damage text" (-10 HP) floats up.
- This creates the *illusion* of a complex firefight, while the server is just doing `HP = HP - Damage`.7

Section 4: Multiplayer & State Synchronization

4.1 Authoritative Server Model

In a competitive game, the client cannot be trusted. If the client says "I moved my tank instanty," the server must reject it.

The "Intent" Pattern:

1. **Client:** Player drags unit.
2. **Client:** Emits `CMD_MOVE_UNIT` { unitId, targetX, targetY }.
3. **Server:** Checks:
   - Does player own unit?
   - Is unit alive?
   - Is destination valid?
   - *Calculation:* Speed is 5 units/sec. Distance is 100 units. Arrival is `Now + 20 seconds`.
4. **Server:** Stores `arrival_timestamp` in Redis. Adds BullMQ Job.
5. **Server:** Broadcasts `UNIT_MOVING` { id, startPos, endPos, startTime, duration } to all clients.
6. **Client:** Receives packet. Starts a "Tween" (animation) that moves the sprite from Start to End over the `duration`.

Why this works for the User: This handles the "Real World Time" requirement. If the user closes the browser and comes back in 10 seconds, the client downloads the state, sees the unit is mid-way, and calculates the correct current position based on the `startTime`.29

4.2 Handling Lag and Latency

Since the game is "slower paced," we don't need "Lag Compensation" (rewinding time) like in Counter-Strike.

Optimistic UI: When the player issues an order, the UI should instantly acknowledge it (play sound, show arrow), but mark it as "Pending" (maybe semi-transparent) until the Server acknowledges. This makes the game feel responsive on mobile networks.36

Section 5: The "Gap" - Gameplay Mechanics

5.1 The "Active" Idle Game

To fill the gap between "weeks" and "minutes," the game needs to encourage frequent short check-ins.

Mechanics:

- **Production Queues:** Short timers (build a tank: 5 mins) encourage staying online. Long timers (build a base: 4 hours) encourage leaving.
- **Push Notifications:** "Your territory is under attack!" (Critical for mobile retention).
- **The "Fog" Risk:** If you don't scout, you don't know if an enemy is massing on your border. This forces players to zoom in and check their perimeters, utilizing the map mechanics.3

5.2 Bot AI (Utility System)

Bots need to play "human-like" over 2 days.

Influence Maps:

Instead of complex pathfinding for every decision, the bot maintains a grid of "Influence."

- Enemy Territory = High Danger.
- Own Territory = Safety.
- Resource Node = High Desire. The bot calculates Desire - Danger for adjacent tiles. If the score is positive, it moves. This is computationally cheap and creates "natural" looking expansion behavior.8

Section 6: AI-Assisted Development Plan (The "File")

This section directly answers the user's request for "a good list of things I can put in a file" to keep Copilot on track.

6.1 `copilot-context.md`

Create this file in your project root.

Xeno-Conquest Project Context

Core Identity

- **Genre:** Persistent Browser MMORTS (24-48h matches).
- **Visual Style:** Top-down 2D, Sci-fi Alien World.
- **Key Mechanic:** Seamless semantic zoom (Map -> Icons -> Animated Sprites).

Tech Stack Rules

- **Frontend:** React (UI Overlay) + Pixi.js (Game View) + Vite.

- **Backend:** Node.js (Express) + Socket.IO + BullMQ (Redis).
- **Language:** Strict TypeScript.
- **State:** Zustand (Client), Redis (Server).

Architecture Patterns

1. **The "Manager" Pattern:** React components do NOT control Pixi Sprites directly. React calls methods on a `GameMapManager` class, which manipulates the Pixi Stage.
2. **Shared Types:** Always define data structures (Unit, Territory) in `packages/shared` before implementing logic.
3. **Job Queue:** Any logic taking > 5 seconds (movement, building) MUST use the BullMQ job queue, not `setTimeout`.
4. **Optimistic UI:** Clients predict successful moves immediately; Server validates asynchronously.

Coding Style

- Use Functional Components for React.
- Use `async/await` for asynchronous operations.
- Prefer `interface` over `type` for object definitions.
- Comment complex math (especially map coordinate transforms).

6.2 Prompting Strategy

When you feel Copilot getting lost:

1. **Stop.** Do not continue the chat thread.
2. **Open a new Chat.**
3. **Paste the Summary:** "I am working on the Xeno-Conquest project. Refer to `copilot-context.md`. I am currently implementing."
4. **Paste the specific file interface:** "Here is the `Unit` interface from `shared/types.ts`. Now, help me write the `moveUnit` function in `server/gameEngine.ts`."

This "Context Reset" is the single most effective way to fix AI hallucinations.

Section 7: Implementation Checklist

This checklist is designed to be copy-pasted into a project management tool (Trello/Jira) or a `TODO.md` file.

Phase 1: Foundation (The Grid)

- [ ] **Setup Monorepo:** Configure `npm workspaces` with `/client`, `/server`, `/shared`.
- [ ] **Shared Types:** Create `packages/shared/src/types.ts` defining `Point`, `Territory`, `Unit`.
- [ ] **Pixi Boilerplate:** Initialize `pixi.js` application in `/client`. Ensure it resizes to full window.
- [ ] **Viewport:** Implement `pixi-viewport`. Test pan/zoom on mobile (touch events).
- [ ] **Map Generation:** Write a function (Voronoi or Perlin Noise) to generate random territories. Render them as `PIXI.Graphics`.

Phase 2: Connectivity (The Socket)

- [ ] **Server Setup:** Initialize Express + Socket.IO in `/server`.
- [ ] **Handshake:** On client connect, server sends the `MapData`. Client renders it.
- [ ] **Redis:** Set up a local Redis instance (Docker is easiest).
- [ ] **State Persistence:** Save the generated map to Redis so it doesn't change on server restart.

Phase 3: The Game Loop (The Ticks)

- [ ] **BullMQ:** Install BullMQ. Create a `movementQueue`.
- [ ] **Movement Command:**
  - Client emits `move` event.
  - Server calculates arrival time.
  - Server adds job to Queue.
  - Server emits `unit_moving` to all clients.
- [ ] **Interpolation:** Client receives `unit_moving`. Create a Pixi Ticker that updates the sprite position every frame based on `(Now - StartTime) / Duration`.

Phase 4: The "Juice" (The Visuals)

- [ ] **Asset Loading:** Load sprite sheets for troops.
- [ ] **LOD Logic:** In the Pixi Ticker, check `viewport.scaled`.
  - If scale < 0.5, set `sprite.visible = false`, `icon.visible = true`.
  - If scale > 0.5, swap them.
- [ ] **Combat visual:** When two enemy units are close (client-side check), trigger `sprite.play('shoot')`.

Phase 5: The Meta (Victory)

- [ ] **Territory Capture:** When movement job finishes, if enemy territory, trigger combat logic.
- [ ] **Victory Condition:** Check if one player owns > 50% of territories.
- [ ] **Reset:** Clear Redis and generate a new map for the next match.

Section 8: Deep Dive - Specific Code Patterns

8.1 The Pixi-React Integration Pattern

Do not use `useEffect` to manage every sprite. It is too slow.

Recommended Pattern:

TypeScript

```
// GameMap.tsx
const GameMap = () => {
    const canvasRef = useRef(null);
    const appRef = useRef<GameApplication | null>(null);

    useEffect(() => {
        // Initialize Pixi App ONCE
        if (!appRef.current) {
            appRef.current = new GameApplication(canvasRef.current);
        }

        return () => appRef.current?.destroy();
    },);

    // React handles the UI, Pixi handles the Canvas
    return <div ref={canvasRef} className="map-container" />;
};
```

The `GameApplication` class (pure TypeScript) then handles the specialized Pixi logic, keeping React "clean" and preventing the AI from mixing the two lifecycles improperly.14

8.2 Handling "Infinite" Coordinates

In a large map, coordinates can get huge. JavaScript numbers are fine, but rendering precision can drift.

- **Chunking:** Divide the world into "Chunks" (e.g., 1000x1000 pixels).

- **Relative Rendering:** When the camera moves, you don't move the camera to `x: 1,000,000`. You move the *World Container* so that the visible chunk is at `0,0`. This keeps GPU precision high (preventing "jitter" on mobile).

Conclusion

The vision of a "Slow RTS" with "Deep Zoom" is feasible and fills a compelling gap in the market. The failure mode of such projects is usually architectural—attempting to synchronize state too frequently or rendering too many DOM elements.

By adhering to the **Tick-Based Server** architecture (using BullMQ) and the **LOD Rendering** architecture (using Pixi.js Quadtrees), the technical risk is minimized. The provided `copilot-context.md` strategy effectively mitigates the "confused AI" problem, turning Copilot from a liability into a highly effective pair programmer for this specific modularized stack. The path forward is to build the *Time Engine* (Backend) and the *Map Engine* (Frontend) as separate, robust systems before bridging them with game logic.